



MCD512-01 — BIG DATA

Integración de IoT y Big Data para mantenimiento predictivo en una planta de manufactura

Propuesta conceptual y prototipo funcional

Presentado por:

Dan Carrión — Matrícula 1131022

Prof. José M. Aquino Cepeda, MSc.

Docente

Maestría en Ciencia de Datos

Instituto Tecnológico de Santo Domingo (INTEC)

Santo Domingo, República Dominicana

Octubre 2026

Contenido

Contenido	2
1. Introducción	3
2. Objetivos	3
3. Arquitectura propuesta	3
3.1 Capa de dispositivos IoT (edge layer)	4
3.2 Capa de transferencia (fog / cloud layer)	4
3.3 Capa de almacenamiento (big data layer)	4
3.4 Capa de procesamiento y análisis	4
3.5 Capa de visualización y acción	5
3.6 Decisiones de diseño	5
4. Despliegue del prototipo	5
5. Capa 1 — Simulación de dispositivos IoT	6
6. Capa 2 — Transferencia MQTT → Kafka	6
7. Capa 3 — Almacenamiento escalable	7
7.1 Data lake sobre HDFS	7
7.2 Serie de tiempo sobre InfluxDB	7
8. Capa 4 — Procesamiento y detección de anomalías	8
8.1 El job de streaming	8
8.2 El modelo de detección	8
8.3 Evaluación del modelo	9
8.4 Ejecución en el prototipo	10
9. Capa 5 — Visualización en Grafana	11
10. Validación de extremo a extremo	12
11. Escalabilidad y consideraciones de producción	13
12. Conclusiones	14
13. Referencias	14
Anexo — Estructura del repositorio	14

1. Introducción

En la Industria 4.0 el valor no está en instalar sensores, sino en lo que se hace con lo que miden. Una planta de manufactura con máquinas rotativas —extrusoras, prensas, compresores, bombas— genera telemetría continua de vibración, temperatura, corriente y velocidad de giro. Ese flujo es pequeño por lectura pero enorme en agregado: cuatro máquinas muestreadas cada dos segundos producen alrededor de 172 mil lecturas diarias, y una planta real con doscientos activos supera los ocho millones.

El problema de negocio que resuelve este trabajo es el mantenimiento. El mantenimiento correctivo detiene la línea sin aviso y el preventivo por calendario reemplaza piezas que todavía tienen vida útil. El mantenimiento predictivo usa la telemetría para anticipar la falla y programar la intervención en la ventana de menor costo. Para lograrlo hacen falta las dos piezas del título: IoT para capturar el dato en el borde y Big Data para transportarlo, almacenarlo a escala y analizarlo.

Este documento presenta el diseño conceptual y un prototipo funcional que implementa la cadena completa, desde la generación de la telemetría hasta el tablero de monitoreo, junto con la evidencia de su ejecución de extremo a extremo.

2. Objetivos

- Diseñar e implementar una arquitectura IoT–Big Data por capas que capture datos de sensores en planta, los transmita eficientemente y los almacene en una plataforma escalable.
- Configurar la cadena de transferencia MQTT → Kafka, justificando el rol de cada protocolo dentro del flujo.
- Implementar almacenamiento dual: HDFS como data lake histórico e InfluxDB como base de series de tiempo para consulta en vivo.
- Aplicar Spark Structured Streaming y un modelo de aprendizaje no supervisado (Isolation Forest) para detectar anomalías en línea.
- Construir un tablero en Grafana que permita monitorear la planta y disparar alertas de mantenimiento.
- Evaluar el prototipo en capacidad de detección, latencia y escalabilidad.

3. Arquitectura propuesta

La solución se organiza en cinco capas. Cada una resuelve un problema distinto y puede escalarse por separado, que es precisamente la razón de separarlas.

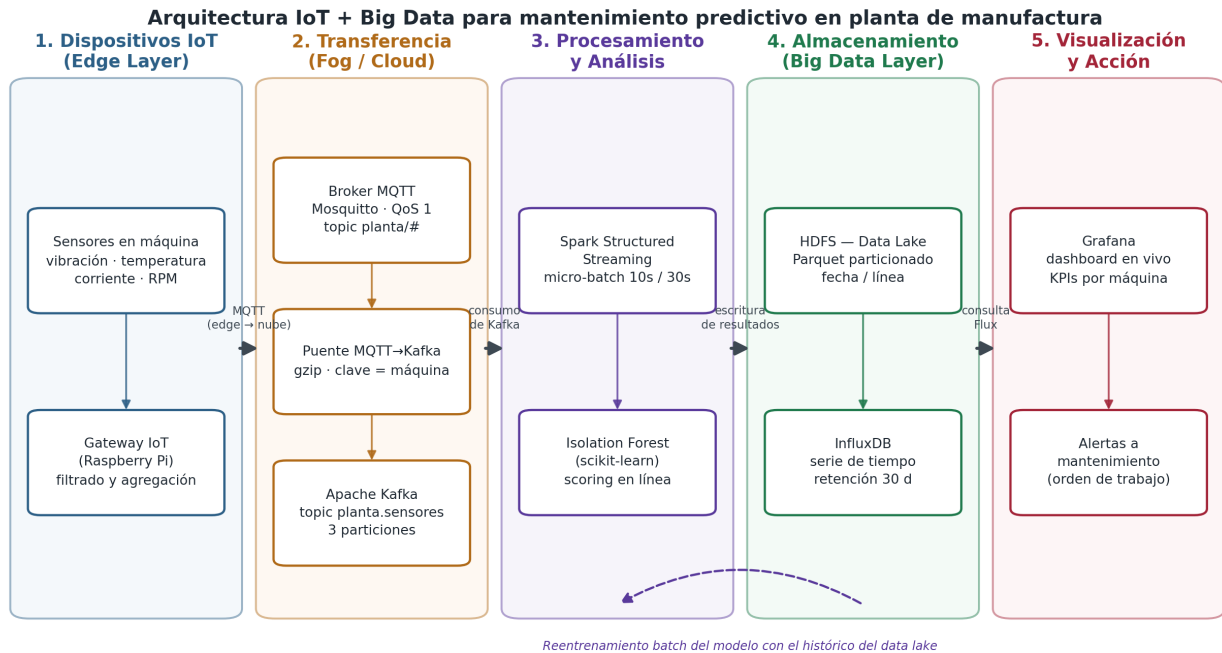


Figura 1 — Arquitectura IoT + Big Data por capas

3.1 Capa de dispositivos IoT (edge layer)

Sensores de vibración, temperatura, corriente y RPM montados sobre cada máquina, conectados a un gateway —una Raspberry Pi en el diseño físico— que filtra ruido, agrega lecturas y publica por MQTT. En el prototipo, sensores y gateway se sustituyen por un simulador en Python que reproduce el comportamiento estadístico de cuatro máquinas reales.

3.2 Capa de transferencia (fog / cloud layer)

MQTT es el protocolo del borde: cabecera de dos bytes, tolerancia a enlaces intermitentes y funcionamiento con poco ancho de banda. Kafka es el bus de la nube: retiene los mensajes, permite que varios consumidores lean el mismo flujo sin interferirse y absorbe picos de ingesta. No compiten entre sí; se complementan. Un proceso puente traduce de un protocolo al otro.

3.3 Capa de almacenamiento (big data layer)

Dos destinos con propósitos distintos. HDFS guarda el dato crudo en formato Parquet, particionado por fecha y línea de producción, como registro inmutable para reentrenamiento de modelos y análisis histórico. InfluxDB guarda la serie de tiempo reciente, optimizada para las consultas por rango que el tablero ejecuta cada diez segundos.

3.4 Capa de procesamiento y análisis

Spark Structured Streaming consume de Kafka y ejecuta dos consultas en paralelo sobre el mismo flujo. La primera vuelca el dato crudo a HDFS; la segunda aplica el modelo de anomalías y escribe el resultado en InfluxDB. Es el patrón de vía caliente y vía fría, que permite optimizar cada camino por separado.

3.5 Capa de visualización y acción

Grafana consulta InfluxDB mediante el lenguaje Flux y presenta la telemetría por máquina, el score de anomalía y el conteo de eventos. Desde el mismo tablero se configuran las alertas que llegan al equipo de mantenimiento y disparan la orden de trabajo.

3.6 Decisiones de diseño

Cada componente se eligió frente a una alternativa razonable. La tabla siguiente resume el criterio aplicado.

Decisión	Alternativa considerada	Criterio de selección
MQTT en el borde	HTTP / REST	Menor overhead por mensaje; QoS y reconexión nativos
Kafka como bus	Escritura directa a la base	Desacopla ingesta de procesamiento; permite reprocesar
Parquet sobre HDFS	JSON o CSV crudo	Formato columnar comprimido: menos espacio, lectura selectiva
Isolation Forest	Umbrales fijos por variable	Detecta combinaciones anómalas sin datos etiquetados
InfluxDB junto a HDFS	Solo HDFS	HDFS no responde con la latencia que exige un tablero en vivo

4. Despliegue del prototipo

Todo el stack corre en contenedores orquestados con Docker Compose sobre un Mac con Apple Silicon. Las imágenes fueron seleccionadas o construidas para arquitectura arm64: HDFS y Spark se construyen sobre un JRE multiplataforma porque las imágenes oficiales de Hadoop solo se publican para amd64.

```
docker compose up -d --build
docker compose run --rm spark python3 /app/ml/entrenar_modelo.py
docker compose restart spark
docker compose ps
```

La Figura 2 muestra los ocho servicios en ejecución. Los cuatro que declaran verificación de salud —HDFS, InfluxDB, Kafka y Mosquitto— aparecen como healthy. Spark registra un tiempo de actividad menor que el resto porque se reinició después de entrenar el modelo, tal como indica el procedimiento.

```

(.venv) dancarrion@192-iot-bigdata-mcd512 % docker compose restart spark
[+] restart 0/1
Container spark Restarting
(.venv) dancarrion@192-iot-bigdata-mcd512 % docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
grafana              grafana/grafana:11.1.0 "/run.sh"              grafana     4 minutes ago  Up 4 minutes  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
hdfs                 iot-bigdata-mcd512-hdfs "/_cacert_entrypoint..." hdfs        4 minutes ago  Up 4 minutes  0.0.0.0:8020->8020/tcp, [::]:8020->8020/tcp, 0.0.0.0:9864->9864/tcp, [::]:9864->9864/tcp, 0.0.0.0:9870->9870/tcp, [::]:9870->9870/tcp
influxdb             influxdb:2.7         "/_cacert_entrypoint..." influxdb     4 minutes ago  Up 4 minutes  0.0.0.0:8086->8086/tcp, [::]:8086->8086/tcp
kafka                apache/kafka:3.7.1   "/_cacert_entrypoint..." kafka        4 minutes ago  Up 4 minutes  0.0.0.0:29092->29092/tcp, [::]:29092->29092/tcp
mosquitto            eclipse-mosquitto:2   "/docker-entrypoint..." mosquitto    4 minutes ago  Up 4 minutes  0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp
puente              iot-bigdata-mcd512-puente "python -u mqtt_kafka..." puente        4 minutes ago  Up 4 minutes  0.0.0.0:4040->4040/tcp, [::]:4040->4040/tcp
simulador            iot-bigdata-mcd512-simulador "python -u simulador..." simulador     4 minutes ago  Up 4 minutes
spark                iot-bigdata-mcd512-spark "/_cacert_entrypoint..." spark        4 minutes ago  Up 25 seconds

```

Figura 2 — Servicios del stack desplegados con docker compose

5. Capa 1 — Simulación de dispositivos IoT

El simulador modela cuatro máquinas con valores nominales distintos y tres regímenes de operación: funcionamiento normal con ruido gaussiano, degradación progresiva de una máquina —MAQ-03, un compresor que empieza a fallar a los cinco minutos— y picos puntuales aleatorios con probabilidad del uno por ciento en todas las máquinas.

La degradación modela la firma típica de un rodamiento en deterioro: vibración, temperatura y corriente en ascenso mientras las RPM caen ligeramente. Cada ciclo publica una lectura por máquina cada dos segundos; el registro de la Figura 3 muestra la conexión al broker y el contador acumulado de lecturas publicadas.

```

(.venv) dancarrion@192-iot-bigdata-mcd512 % docker compose logs -f simulador
simulador | [simulador] Conectado a MQTT mosquitto:1883
simulador | [simulador] 100 lecturas publicadas (0.0 min de operacion)
simulador | [simulador] 200 lecturas publicadas (1.6 min de operacion)
simulador | [simulador] 300 lecturas publicadas (2.5 min de operacion)
simulador | [simulador] 400 lecturas publicadas (3.2 min de operacion)
simulador | [simulador] 500 lecturas publicadas (4.1 min de operacion)
simulador | [simulador] 600 lecturas publicadas (5.0 min de operacion)
simulador | [simulador] 700 lecturas publicadas (5.8 min de operacion)
simulador | [simulador] 800 lecturas publicadas (6.6 min de operacion)

```

Figura 3 — Simulador conectado a Mosquitto y publicando lecturas

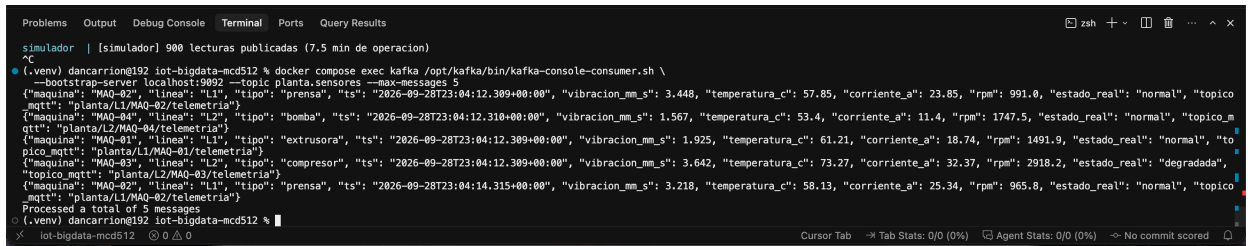
6. Capa 2 — Transferencia MQTT → Kafka

El broker Mosquitto recibe las publicaciones en tópicos jerárquicos con el patrón planta/línea/máquina/telemetry, usando QoS 1, que garantiza al menos una entrega. El puente se suscribe al comodín planta/# y reenvía cada mensaje a Kafka empleando el identificador de máquina como clave de partición, de modo que todas las lecturas de una misma máquina conserven su orden.

Dos ajustes de eficiencia merecen mención en el productor de Kafka: el parámetro `linger_ms` agrupa mensajes en micro-lotes en lugar de hacer un viaje de red por lectura, y la compresión `gzip` reduce el volumen transferido, que en un despliegue real sobre enlace celular se traduce en costo directo.

La Figura 4 muestra cinco mensajes leídos directamente del tópico `planta.sensores`. Cada uno conserva el tópico MQTT de origen, lo que permite trazar la lectura hasta el dispositivo. El mensaje de MAQ-03 ya

aparece con estado degradada, con 3.64 mm/s de vibración y 73.3 °C frente a sus valores nominales de 2.8 mm/s y 70 °C.



```
simulador | [simulador] 900 lecturas publicadas (7.5 min de operacion)
(.venv) dancarrion@192 iot-bigdata-mcd512 % docker compose exec kafka /opt/kafka/bin/kafka-console-consumer.sh \
--bootstrap-server localhost:9092 --topic planta.sensores --max-messages 5
{"maquina": "MAQ-02", "linea": "L1", "tipo": "prensa", "ts": "2026-09-28T23:04:12.309+00:00", "vibracion_mm_s": 3.448, "temperatura_c": 57.85, "corriente_a": 23.85, "rpm": 991.0, "estado_real": "normal", "topico_mqtt": "planta/L1/MAQ-02/telemetry"}
{"maquina": "MAQ-04", "linea": "L2", "tipo": "bomba", "ts": "2026-09-28T23:04:12.310+00:00", "vibracion_mm_s": 1.567, "temperatura_c": 53.4, "corriente_a": 11.4, "rpm": 1747.5, "estado_real": "normal", "topico_mqtt": "planta/L2/MAQ-04/telemetry"}
{"maquina": "MAQ-01", "linea": "L1", "tipo": "extrusora", "ts": "2026-09-28T23:04:12.309+00:00", "vibracion_mm_s": 1.925, "temperatura_c": 61.21, "corriente_a": 18.74, "rpm": 1491.9, "estado_real": "normal", "topico_mqtt": "planta/L1/MAQ-01/telemetry"}
{"maquina": "MAQ-03", "linea": "L2", "tipo": "compresor", "ts": "2026-09-28T23:04:12.309+00:00", "vibracion_mm_s": 3.642, "temperatura_c": 73.27, "corriente_a": 32.37, "rpm": 2918.2, "estado_real": "degradada", "topico_mqtt": "planta/L2/MAQ-03/telemetry"}
{"maquina": "MAQ-02", "linea": "L1", "tipo": "prensa", "ts": "2026-09-28T23:04:14.315+00:00", "vibracion_mm_s": 3.218, "temperatura_c": 58.13, "corriente_a": 25.34, "rpm": 965.8, "estado_real": "normal", "topico_mqtt": "planta/L1/MAQ-02/telemetry"}
Processed a total of 5 messages
(.venv) dancarrion@192 iot-bigdata-mcd512 %
```

Figura 4 — Mensajes JSON consumidos del tópico *planta.sensores* en Kafka

7. Capa 3 — Almacenamiento escalable

7.1 Data lake sobre HDFS

Spark escribe el dato crudo en `hdfs://hdfs:8020/datalake/crudo/telemetry`, particionado por fecha y línea. El particionado permite que una consulta sobre un día o una línea concretos lea únicamente ese directorio en lugar de recorrer el histórico completo, técnica conocida como *partition pruning*. El formato Parquet añade compresión y lectura por columnas.

La Figura 5 muestra la estructura resultante: dentro de la partición `fecha=2026-09-28` aparecen las subparticiones `linea=L1` y `linea=L2`. Que los directorios marquen 0 B es normal, porque HDFS no agrega el tamaño de su contenido; los archivos Parquet están dentro de cada subpartición.

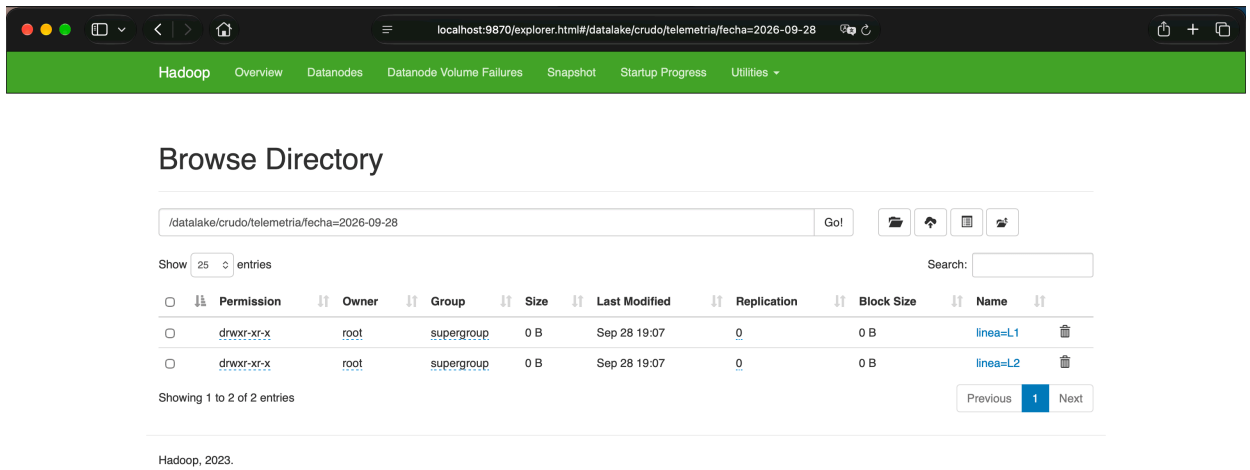


Figura 5 — Explorador del NameNode con el data lake particionado por fecha y línea

7.2 Serie de tiempo sobre InfluxDB

InfluxDB recibe cada lectura enriquecida con el resultado del modelo. El esquema usa máquina, línea y tipo como tags indexados, que son los campos por los que se filtra, y las variables físicas junto al score de anomalía como fields. La política de retención se fijó en treinta días: pasado ese plazo el dato vive únicamente en el data lake, que resulta considerablemente más barato por gigabyte almacenado.

8. Capa 4 — Procesamiento y detección de anomalías

8.1 El job de streaming

El job levanta dos consultas sobre el mismo `readStream` de Kafka. El esquema se declara de forma explícita porque en streaming no puede inferirse, y el parámetro `maxOffsetsPerTrigger` limita cuántos registros entran por micro-batch, que es el mecanismo de contrapresión cuando la ingesta supera la capacidad de procesamiento.

8.2 El modelo de detección

Se emplea Isolation Forest, un método no supervisado que aísla observaciones raras mediante árboles de partición aleatoria. La elección responde a la realidad de una planta: casi nunca existen datos etiquetados de fallas, porque las fallas son escasas y no siempre quedan registradas con precisión temporal.

El detalle metodológico relevante es la construcción de las variables de entrada. Las cuatro máquinas operan en rangos muy distintos —una bomba a 1.6 mm/s y una prensa a 3.4 mm/s son ambas normales—, de modo que en lugar de los valores absolutos se utilizan razones contra el valor nominal de cada máquina. Esa normalización permite que un solo modelo cubra activos heterogéneos.

El modelo se entrena una sola vez, de forma offline, y se guarda en `ml/modelo_anomalias.joblib`. El job de Spark no entrena: carga ese archivo al arrancar y lo aplica a cada micro-batch. Esta separación entre entrenamiento e inferencia es el patrón habitual en producción, porque entrenar dentro del flujo sería costoso y haría que el modelo cambiara continuamente. El entrenamiento se ejecuta dentro del mismo contenedor de Spark para garantizar la misma versión de scikit-learn en ambas etapas.

La Figura 6 documenta ese comportamiento: antes del entrenamiento, el job se detiene y reintenta indicando que el modelo no existe; una vez entrenado, el registro confirma la carga del modelo y el arranque de Spark 3.5.1 sobre arquitectura aarch64.

```
(.venv) dancarrion@192-iot-biddata-mcd512 % docker compose logs -f spark
spark | [spark] No existe el modelo en /app/ml/modelo_anomalias.joblib. Ejecuta primero: docker compose run --rm spark python3 /app/ml/entrenar_modelo.py
spark | 26/09/28 22:56:09 INFO ShutdownHookManager: Shutdown hook called
spark | 26/09/28 22:56:09 INFO ShutdownHookManager: Deleting directory /tmp/spark-a2c85e72-6a47-4919-a02a-594c1099d02b
spark | [spark] No existe el modelo en /app/ml/modelo_anomalias.joblib. Ejecuta primero: docker compose run --rm spark python3 /app/ml/entrenar_modelo.py
spark | 26/09/28 22:56:10 INFO ShutdownHookManager: Shutdown hook called
spark | 26/09/28 22:56:10 INFO ShutdownHookManager: Deleting directory /tmp/spark-04babf89-8720-490e-915e-4ba958519205
spark | 26/09/28 22:56:11 INFO ShutdownHookManager: Shutdown hook called
spark | 26/09/28 22:56:11 INFO ShutdownHookManager: Deleting directory /tmp/spark-511ffa0b-0b88-46de-87bf-d9b72a2ed27f
spark | [spark] No existe el modelo en /app/ml/modelo_anomalias.joblib. Ejecuta primero: docker compose run --rm spark python3 /app/ml/entrenar_modelo.py
spark | 26/09/28 22:56:13 INFO ShutdownHookManager: Shutdown hook called
spark | 26/09/28 22:56:13 INFO ShutdownHookManager: Deleting directory /tmp/spark-fc4022ed-1d33-437d-b7a5-bc525227c247
spark | 26/09/28 22:56:14 INFO ShutdownHookManager: Shutdown hook called
spark | 26/09/28 22:56:14 INFO ShutdownHookManager: Deleting directory /tmp/spark-f236da1d-0203-41f3-867f-7ac9306ea41a
spark | [spark] No existe el modelo en /app/ml/modelo_anomalias.joblib. Ejecuta primero: docker compose run --rm spark python3 /app/ml/entrenar_modelo.py
spark | 26/09/28 22:56:17 INFO ShutdownHookManager: Shutdown hook called
spark | 26/09/28 22:56:17 INFO ShutdownHookManager: Deleting directory /tmp/spark-5ad7be77-7017-4b97-bde2-d4d90ccc04d9
spark | [spark] No existe el modelo en /app/ml/modelo_anomalias.joblib. Ejecuta primero: docker compose run --rm spark python3 /app/ml/entrenar_modelo.py
spark | 26/09/28 22:56:21 INFO ShutdownHookManager: Shutdown hook called
spark | 26/09/28 22:56:21 INFO ShutdownHookManager: Deleting directory /tmp/spark-67361577-95c7-48a4-ac6f-6fd3da064841
spark | [spark] No existe el modelo en /app/ml/modelo_anomalias.joblib. Ejecuta primero: docker compose run --rm spark python3 /app/ml/entrenar_modelo.py
spark | 26/09/28 22:56:28 INFO ShutdownHookManager: Shutdown hook called
spark | 26/09/28 22:56:28 INFO ShutdownHookManager: Deleting directory /tmp/spark-9c74a911-781b-47f5-9e04-514d5b7e2eba
spark | [spark] No existe el modelo en /app/ml/modelo_anomalias.joblib. Ejecuta primero: docker compose run --rm spark python3 /app/ml/entrenar_modelo.py
spark | 26/09/28 22:56:42 INFO ShutdownHookManager: Shutdown hook called
spark | 26/09/28 22:56:42 INFO ShutdownHookManager: Deleting directory /tmp/spark-efe5df88-c1bf-453c-adb8-93b530130083
spark | [spark] No existe el modelo en /app/ml/modelo_anomalias.joblib. Ejecuta primero: docker compose run --rm spark python3 /app/ml/entrenar_modelo.py
spark | 26/09/28 22:57:09 INFO ShutdownHookManager: Shutdown hook called
spark | 26/09/28 22:57:09 INFO ShutdownHookManager: Deleting directory /tmp/spark-05ac18bb-1825-4e66-8a35-d3ccba56ad72
spark | [spark] Modelo cargado desde /app/ml/modelo_anomalias.joblib
spark | 26/09/28 22:58:01 INFO SparkContext: Running Spark version 3.5.1
spark | 26/09/28 22:58:01 INFO SparkContext: OS info Linux, 7.0.14-linuxkit, aarch64
spark | 26/09/28 22:58:01 INFO SparkContext: Java version 17.0.20.1
```

Figura 6 — Arranque del job de Spark: reintentos sin modelo y carga tras el entrenamiento

8.3 Evaluación del modelo

El modelo se evaluó sobre 4,200 lecturas simuladas: 3,000 de operación normal de las cuatro máquinas y 1,200 de MAQ-03 en degradación avanzada. Como el simulador conoce el estado real de cada lectura, es posible calcular una matriz de confusión, algo que en producción no sería posible.

Estado real	Predicho normal	Predicho degradada
Normal	2,930	70
Degradada	0	1,200

Métrica	Clase normal	Clase degradada
Precisión	1.000	0.945
Recall	0.977	1.000
F1-score	0.988	0.972
Soporte	3,000	1,200

El modelo recupera la totalidad de las lecturas degradadas a costa de marcar el 2.3 % de las sanas. Ese porcentaje no es enteramente error: corresponde en buena medida a los picos puntuales que el simulador inyecta en máquinas sanas, que son anomalías legítimas aunque no indiquen una falla en desarrollo. La Figura 7 muestra la separación entre ambos regímenes en la distribución del score.

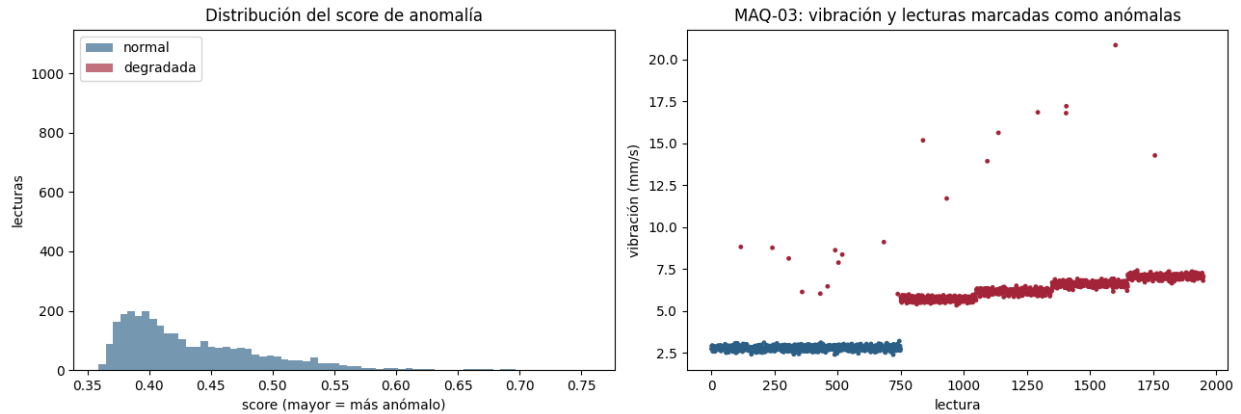


Figura 7 — Distribución del score y lecturas de MAQ-03 marcadas como anómalas

8.4 Ejecución en el prototipo

En operación, cada micro-batch de diez segundos procesa 20 lecturas: cuatro máquinas publicando cada dos segundos. La Figura 8 muestra batches con 5 anomalías de forma sostenida, que es exactamente el número de lecturas de una sola máquina por batch: el modelo está marcando todas las lecturas de MAQ-03. Cuando aparece una sexta, corresponde a un pico aleatorio de otra máquina.

```
spark | [spark] batch 55: 20 lecturas, 5 anomalías
spark | [spark] batch 56: 20 lecturas, 5 anomalías
spark | [spark] batch 57: 20 lecturas, 5 anomalías
spark | [spark] batch 58: 20 lecturas, 5 anomalías
spark | [spark] batch 59: 20 lecturas, 5 anomalías
spark | [spark] batch 60: 20 lecturas, 5 anomalías
spark | [spark] batch 61: 20 lecturas, 5 anomalías
spark | [spark] batch 62: 20 lecturas, 6 anomalías
spark | [spark] batch 63: 20 lecturas, 5 anomalías
spark | [spark] batch 64: 20 lecturas, 6 anomalías
spark | [spark] batch 65: 20 lecturas, 6 anomalías
spark | [spark] batch 66: 20 lecturas, 5 anomalías
spark | [spark] batch 67: 20 lecturas, 6 anomalías
spark | [spark] batch 68: 20 lecturas, 5 anomalías
spark | [spark] batch 69: 20 lecturas, 6 anomalías
```

Figura 8 — Micro-batches de scoring: 20 lecturas y 5–6 anomalías por batch

La Figura 9 confirma que las dos consultas de streaming corren en paralelo. La tasa de entrada promedio, cercana a 2 lecturas por segundo, coincide con la frecuencia de publicación del simulador. La consulta de scoring acumula tres veces más batches que la de HDFS porque sus disparadores son de 10 y 30 segundos respectivamente.

Streaming Query

Active Streaming Queries (2)

Name	Status	ID	Run ID	Start Time	Duration	Avg Input/sec	Avg Process/sec	Latest Batch
scoring_a_influxdb	RUNNING	7c0f440f-e4c5-42ea-9f70-ebc1fe478e84	076fd9b2-4a74-45ad-94a6-a0e93b879a95	2026/09/28 23:00:03	10 minutes 3 seconds	1.99	19.24	72
crudo_a_hdfs	RUNNING	7436f974-879a-4537-93a0-2fe0daaa6d38	1ca9cf8b-df59-44c9-9e51-5d6ef03de676	2026/09/28 23:00:03	10 minutes 3 seconds	1.92	27.58	24

Figura 9 — Spark UI: consultas `scoring_a_influxdb` y `crudo_a_hdfs` en ejecución

9. Capa 5 — Visualización en Grafana

El tablero se aprovisiona automáticamente al levantar el stack. Contiene seis paneles: vibración y temperatura por máquina, conteo total de anomalías, anomalías por máquina, evolución del score y un panel combinado de corriente y RPM. El refresco es de diez segundos, alineado con el disparador del micro-batch de scoring.

En la Figura 10 se observa la falla desarrollándose: la vibración de MAQ-03 sube de forma sostenida desde unos 2.8 mm/s hasta cerca de 7 mm/s, y su temperatura pasa de 70 °C a casi 87 °C, mientras las otras tres máquinas se mantienen estables.

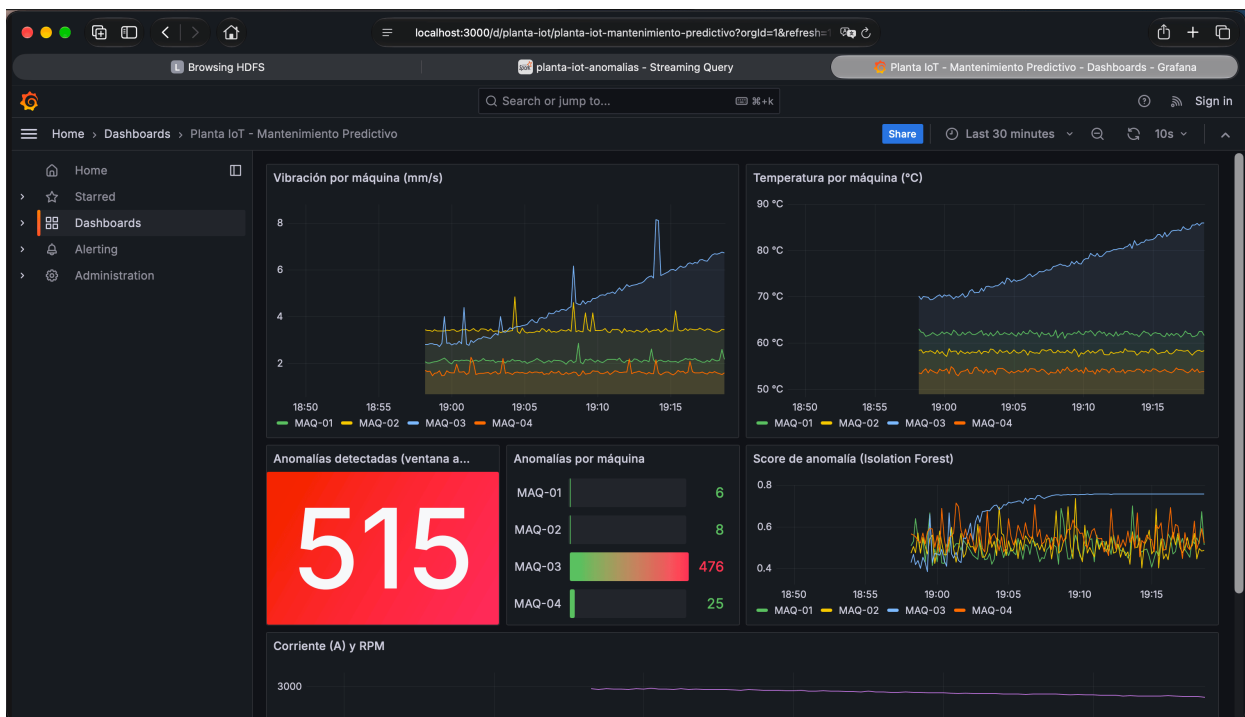


Figura 10 — Tablero de monitoreo con la degradación de MAQ-03 en curso

La Figura 11 pone el foco en la detección. De 595 anomalías en la ventana, 551 corresponden a MAQ-03; las demás máquinas no superan las 27, todas atribuibles a picos aislados. MAQ-04 acumula algo más que MAQ-01 y MAQ-02 porque su vibración nominal es la más baja, de modo que un pico se aleja proporcionalmente más de su valor normal. El score de MAQ-03 se estabiliza cerca de 0.76: una vez que las lecturas quedan muy lejos de lo visto en el entrenamiento, el bosque las aísla en el mínimo de particiones posible y el score deja de crecer.

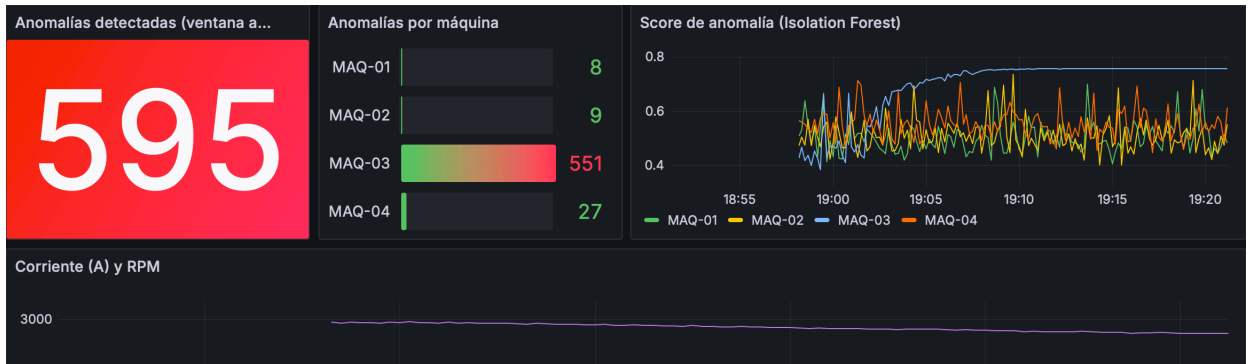


Figura 11 — Paneles de detección: conteo, anomalías por máquina y score

10. Validación de extremo a extremo

Para cerrar el ciclo se leyó lo que efectivamente quedó almacenado. Del data lake en HDFS se recuperaron 3,258 lecturas correspondientes a unos 27 minutos de operación, repartidas de forma homogénea entre las cuatro máquinas:

Línea	Máquina	Lecturas	Vibración media	Vibración máx.	Temp. media
L1	MAQ-01	822	2.12 mm/s	6.17 mm/s	61.99 °C
L1	MAQ-02	822	3.45 mm/s	11.03 mm/s	58.05 °C
L2	MAQ-03	807	5.21 mm/s	19.37 mm/s	79.38 °C
L2	MAQ-04	807	1.62 mm/s	4.62 mm/s	53.97 °C

MAQ-03 es la única máquina cuya media se aparta de su valor nominal: 5.21 mm/s de vibración frente a 2.8 y 79.4 °C frente a 70. Los máximos elevados de MAQ-02 y MAQ-01 corresponden a picos puntuales, que no desplazan la media.

Desde InfluxDB, la consulta del conteo de anomalías de las últimas dos horas arrojó la siguiente distribución:

Máquina	Anomalías detectadas	% del total
MAQ-01	10	1.2 %
MAQ-02	17	2.0 %

MAQ-03	780	93.3 %
MAQ-04	29	3.5 %

La Figura 12 reconstruye la vibración media desde la serie de tiempo. Sus horas están en UTC, mientras que Grafana muestra la hora local de República Dominicana (UTC-4); ambas describen el mismo intervalo. La curva de MAQ-03 se aplana al final porque el simulador lleva la degradación a su máximo en el minuto 25 y la mantiene.

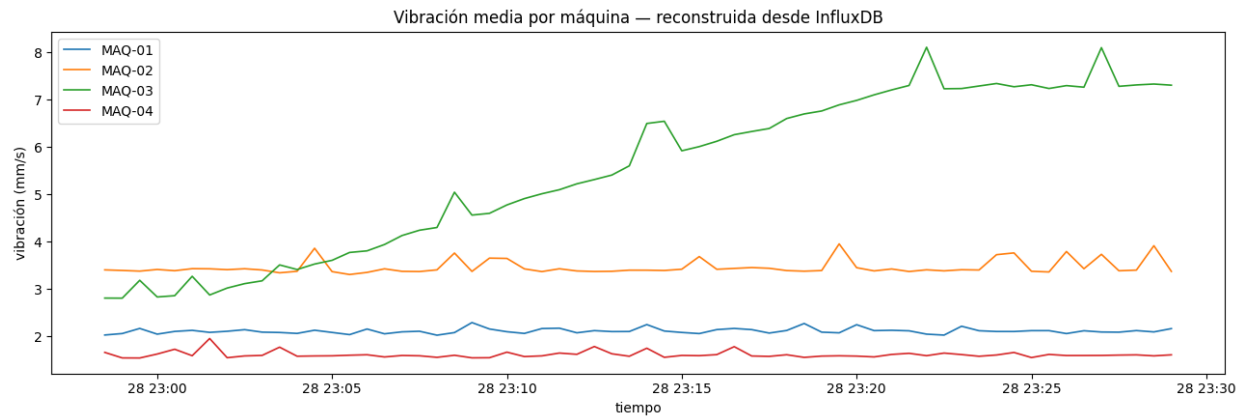


Figura 12 — Vibración media por máquina reconstruida desde InfluxDB

11. Escalabilidad y consideraciones de producción

El prototipo corre en un solo nodo, pero cada componente fue elegido por su trayectoria de escalamiento.

Componente	En el prototipo	En producción
Kafka	1 broker, 3 particiones	Clúster de 3+ brokers; particiones = paralelismo de consumo
HDFS	1 NameNode, 1 DataNode, replicación 1	Varios DataNodes con replicación 3, o S3/ADLS como data lake
Spark	Modo local[2]	Clúster YARN o Kubernetes con autoescalado de ejecutores
InfluxDB	Instancia única, 30 días	Clúster con downsampling automático por niveles
Modelo	Uno global normalizado al nominal	Uno por familia de activo, reentrenado periódicamente

Tres puntos que un despliegue real exige y que este prototipo deliberadamente no cubre. Primero, la seguridad: MQTT y Kafka corren sin autenticación ni cifrado, mientras que en planta se requeriría al menos TLS mutuo en el borde y SASL en el bus. Segundo, el almacenamiento por niveles: mantener el dato reciente en SSD y el histórico en almacenamiento frío reduce el costo de forma sustancial. Tercero, la

deriva del modelo: cambios de producto, de turno o de materia prima desplazan el régimen considerado normal, por lo que hace falta monitorear la tasa de anomalías y reentrenar periódicamente contra el data lake, que es la realimentación representada en el diagrama de arquitectura.

12. Conclusiones

El prototipo demuestra la cadena completa que plantea la propuesta conceptual: dispositivos IoT generando telemetría, transferencia eficiente mediante MQTT y Kafka, almacenamiento escalable en un data lake sobre HDFS más una base de series de tiempo, procesamiento en tiempo real con Spark, detección de anomalías con aprendizaje no supervisado y visualización operativa en Grafana.

La primera conclusión es que IoT sin Big Data es solo instrumentación: el valor aparece cuando el flujo se almacena de forma consultable y se analiza sistemáticamente. La segunda es que la separación en vía caliente y vía fría no constituye redundancia sino diseño, porque HDFS e InfluxDB responden preguntas distintas y ninguna sustituye a la otra. La tercera es que el cuello de botella del mantenimiento predictivo rara vez es el modelo: el Isolation Forest resultó la pieza más simple, mientras que la ingesta confiable, el particionado y la normalización entre activos heterogéneos concentraron la mayor parte del esfuerzo de diseño.

En términos cuantitativos, el detector alcanzó un recall de 1.000 y una precisión de 0.945 sobre la clase degradada en la evaluación controlada. En la ejecución real, el 93.3 % de las anomalías registradas correspondió a la máquina que efectivamente se estaba degradando, con una latencia de extremo a extremo por debajo de los quince segundos entre la lectura del sensor y su aparición en el tablero.

13. Referencias

- Apache Software Foundation. Structured Streaming Programming Guide, Spark 3.5.
- Apache Software Foundation. HDFS Architecture, Hadoop 3.3.
- Apache Software Foundation. Kafka Documentation — Design and Producer Configs.
- OASIS. MQTT Version 3.1.1 Specification.
- Liu, F. T., Ting, K. M. y Zhou, Z. (2008). Isolation Forest. IEEE International Conference on Data Mining.
- InfluxData. InfluxDB 2.7 Documentation — Flux query language.
- Aquino Cepeda, J. M. Propuesta Conceptual: Integración de IoT con Big Data. MCD512-01, INTEC.

Anexo — Estructura del repositorio

Link a GitHub: <https://github.com/dcarrioncordones/iot-bigdata-mcd512>

Nota: El notebook ejecutable, con el código de validación y las salidas de la corrida, se encuentra en la carpeta *notebooks/* del repositorio.

<code>docker-compose.yml</code>	Orquestación de los 8 servicios
<code>simulador/</code>	Dispositivos IoT simulados (MQTT)
<code>punte/</code>	Puente MQTT → Kafka
<code>spark/app/</code>	Job de Structured Streaming + scoring
<code>ml/entrenar_modelo.py</code>	Entrenamiento del Isolation Forest
<code>hadoop/</code>	Imagen HDFS multiarquitectura
<code>grafana/</code>	Datasource y dashboard aprovisionados
<code>notebooks/</code>	Informe técnico ejecutable
<code>capturas/</code>	Evidencia de ejecución
<code>docs/</code>	Diagrama e informe en Word